What is GraphRAG with Neo4j?

Knowledge Graph Retrieval-Augmented Generation • Architecture & Implementation Guide

Traditional RAG (Retrieval-Augmented Generation) converts policy documents into text chunks, turns them into numbers (vectors), and looks for paragraphs that *'sound similar'* to the user's question.

The Problem with Traditional Vector RAG:

In legal and HR compliance, *'similar-sounding'* is not enough. If an employee asks:

> ***"I worked 11 hours today. How much overtime do I get under Maharashtra law?"***

A standard vector search might return random mentions of overtime, but it doesn't understand the relationship between:

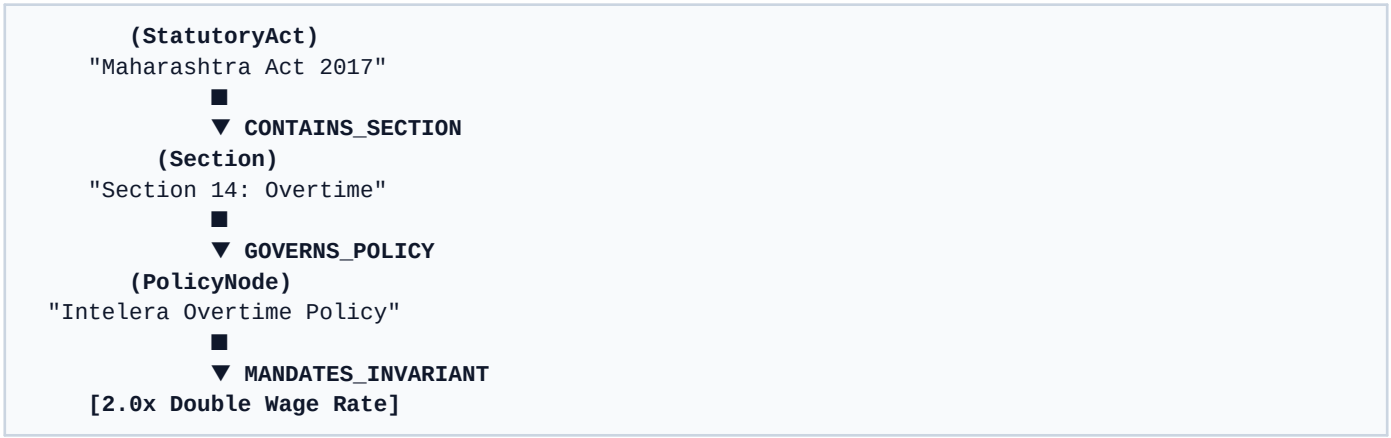
- **The Act:** Maharashtra Shops & Establishments Act 2017
- **Section 13:** Daily work limit is 9 hours
- **Section 14:** Hours beyond 9 mandate 2.0x double hourly pay
- **Company Overtime Policy:** Must be pre-approved by the manager

The GraphRAG Solution:

GraphRAG connects the dots as a web of facts (a Knowledge Graph in Neo4j). Instead of just searching text, the system traverses exact relationships between legal sections, statutory acts, and company policies to produce deterministic, non-hallucinated answers.

1. The Schema (Nodes & Relationships in Neo4j)

Our Neo4j Aura cloud database stores an ontology graph with **18 Nodes** and **16 Relationships**:



Node Types (What exists):

- **StatutoryAct:** The governing legal framework (e.g., *Maharashtra Shops & Establishments Act 2017*, *DPDP Act 2023*, *Maternity Benefit Act 1961*).
- **Section:** The exact legal clauses (e.g., *Section 18: Leave Accrual & Accumulation*, *Section 13: Standard Hours*, *Section 14: Overtime*).
- **PolicyNode:** Company-specific handbooks (e.g., *Earned Leave Policy*, *Sick Leave Policy*, *Health Benefits Plan*).

Relationship Types (How they connect):

- **CONTAINS_SECTION:** An Act contains specific statutory sections.
- **GOVERNS_POLICY:** A statutory law governs a company internal policy.
- **MANDATES_INVARIANT:** A statutory rule that cannot be broken (e.g., minimum 1 day leave per 20 days worked, overtime at 2.0x).
- **REQUIRES_DOCUMENT:** (e.g., Sick leave > 2 days → requires medical certificate from registered doctor).

2. Entity Extraction Logic (How Queries are Understood)

When a user types a question in the chat:

- **Regex & Keyword Scanning:** Fast regex matches statutory terms like Section 18, Section 14, overtime, sick leave, Aadhaar.
- **Entity Mapper (entity_extractor.py):** Maps the user's terms to canonical node IDs in the graph (e.g., "sick leave" → sec_sick_leave_cert + intelera_sick_leave_policy).
- **Multi-Hop Traversal:** Once the starting node is found, it traverses 1 or 2 hops outwards to pull connected statutory rules, caps, and required documents.
- **Context Synthesis:** Combines the graph path into clear facts injected directly into the Policy Agent's prompt.

3. Which Libraries Are We Using?

Library	Role in the System
`neo4j`	Official Python driver connecting via TLS (neo4j+s://) to our Neo4j Aura Cloud cluster to execute Cypher queries.
`networkx`	Fast in-memory Python graph library used as an ultra-fast local cache for instant multi-hop traversals.
`chromadb`	Local vector database for dense semantic embeddings.
`rank-bm25`	BM25 sparse keyword search algorithm.
`pydantic (v2)`	Validates schemas (NodeSchema, RelationshipSchema, GraphTraversalResult).

4. The 3-Way Hybrid Retrieval Pipeline

When the Policy Agent answers a question, it queries three engines concurrently:

- **Neo4j / NetworkX Graph** → Provides the exact legal sections, caps, and relationships.
- **ChromaDB Vector Search** → Finds semantically relevant policy passages.
- **BM25 Keyword Search** → Matches exact policy phrases.

Result Synthesis: The results from all three engines are merged using **Reciprocal Rank Fusion (RRF)** to deliver a grounded, non-hallucinated response with authoritative statutory legal citations.